6. Design Uber/Lyft Ride-Sharing Platform

0) Requirements & Sizing

Functional

- **Real-time location tracking**: Drivers update location every 2-5 seconds
- **Ride matching**: Match riders with nearest available drivers
- **ETA estimation**: Accurate pickup and dropoff time predictions
- **Surge pricing**: Dynamic pricing based on supply/demand
- **Payments**: Pre-authorization, capture after ride, driver payouts
- **Ride lifecycle**: Request → Match → Pickup → In-progress → Complete
- **Notifications**: Push notifications, SMS fallback
- **Fraud detection**: GPS spoofing, fake rides, collusion detection

Non-functional

Metric	Target
Scale	**10M rides/day**, 500+ cities
Online drivers	**1-2M concurrent** drivers sending location
Location write rate	**400K-1M writes/sec** (2-5 sec intervals)
Matching latency	**< 5 seconds** to find a driver
ETA accuracy	**±2-3 minutes** in normal traffic
Payment consistency	**Strongly consistent** (CP)
Availability	**99.9%** for ride services
Global	Multi-region, per-city isolation

Capacity

```
Location Updates:
- 2M drivers × 1 update/3 sec = 667K writes/sec
- Each update: ~200 bytes (driver_id, lat, lng, timestamp, heading, speed)
- Hot data: 2M × 200B = 400MB in Redis GEO
```

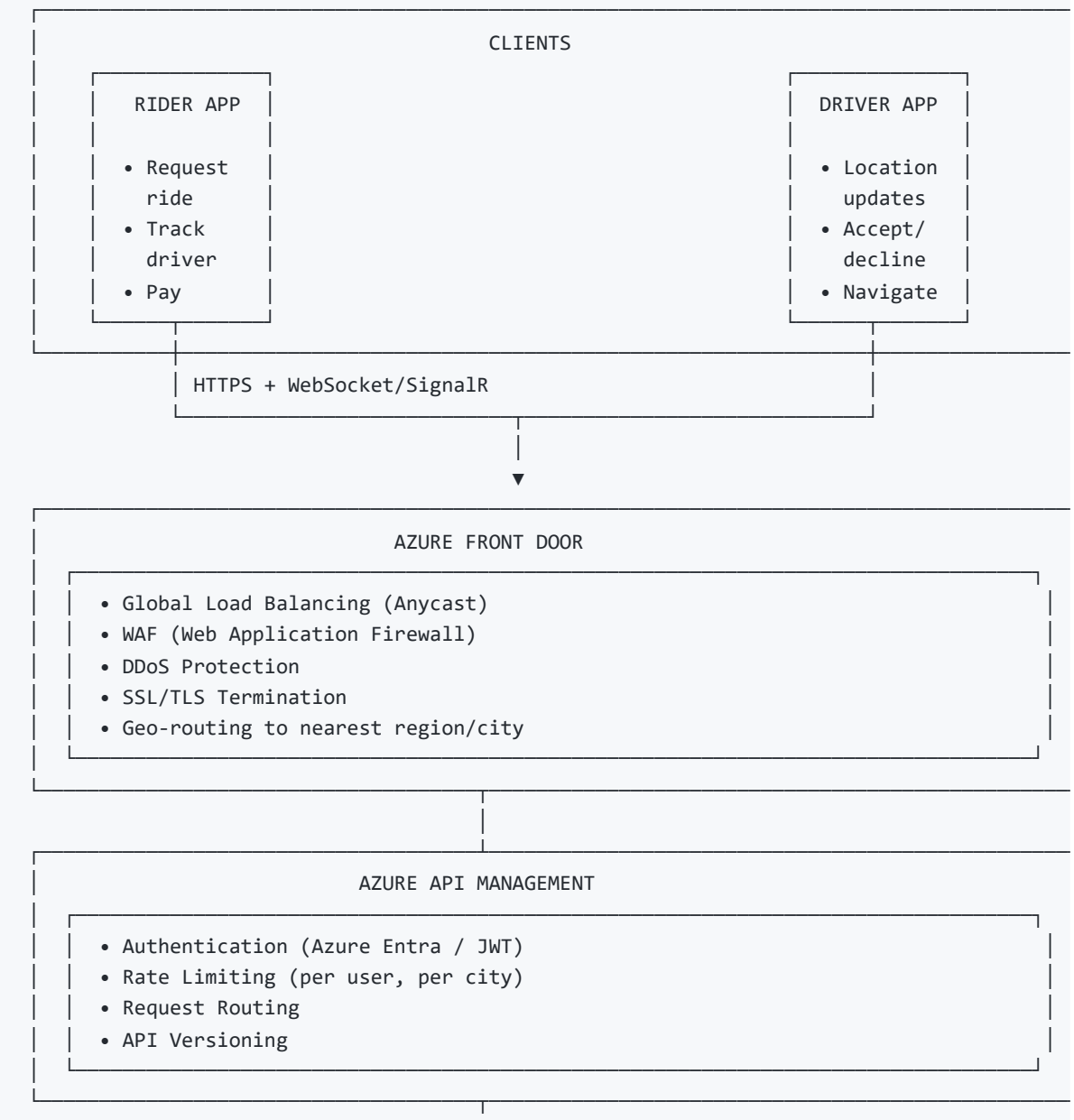
```
Rides:
- 10M rides/day = ~116 rides/sec average
- Peak: 10x = 1,160 rides/sec
- Each ride record: ~2KB

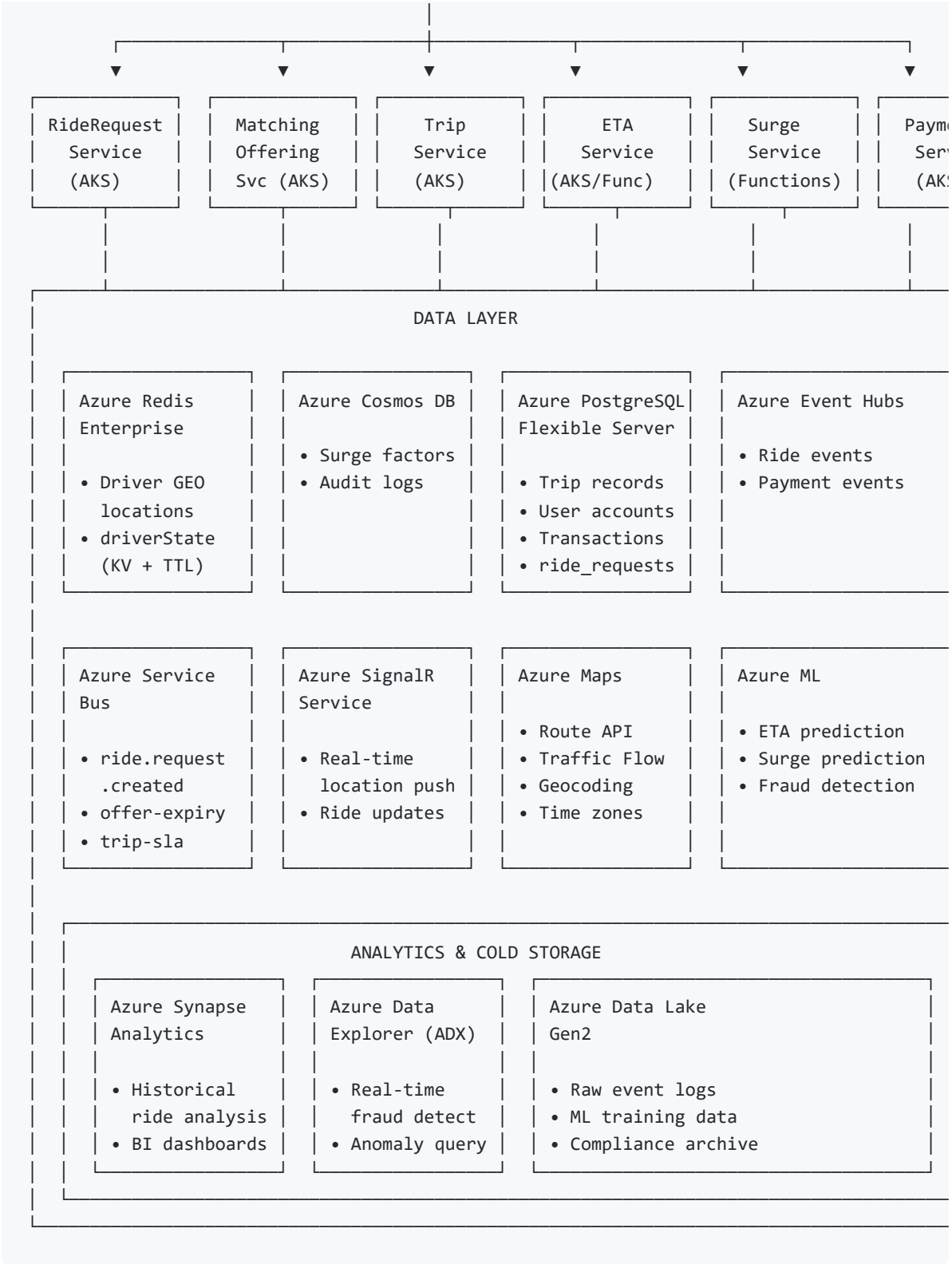
Storage:
- Redis (hot): ~500MB per city cluster
- PostgreSQL (rides): 10M × 2KB × 365 = ~7TB/year
- Analytics (cold): Petabytes over time
```

---

1) High-Level Architecture

Complete Azure Architecture





Components Summary

Component	Azure Service	Purpose
Global Entry	**Azure Front Door**	Anycast DNS, WAF, geo-routing
API Gateway	**Azure API Management**	Auth, rate limiting, routing

Compute	**Azure Kubernetes Service (AKS)**	Microservices per city
Ride Ingestion	**RideRequestService (AKS)**	Persist request, produce to Service Bus
Matching	**MatchingOfferingService (AKS)**	Wave-based driver offering, CAS state
Trip Management	**TripService (AKS)**	Trip persistence, state machine
Pricing	**PricingService (AKS)**	Fare estimate & finalization
Real-time Location	**Azure Redis Enterprise**	GEO index, driver state (KV+Geo, TTL)
Event Streaming	**Azure Event Hubs**	Ride, payment events
Driver Dispatch	**Azure Service Bus**	ride.request.created, offer-expiry, trip-sla-timeout (scheduled)
Real-time Push	**Azure SignalR Service**	WebSocket to riders/drivers
Routing & Maps	**Azure Maps**	Route API, traffic, ETA
ML Models	**Azure ML Managed Endpoints**	ETA adjustment, fraud detection
Surge Compute	**Azure Functions**	Serverless surge calculation
Ride Database	**Azure PostgreSQL**	ACID transactions for rides
Document Store	**Azure Cosmos DB**	Driver status, surge factors
Notifications	**Azure Notification Hubs**	APNs + FCM push
Analytics	**Azure Synapse + ADX**	Real-time and batch analytics

---

API Design

WebSocket Events

Driver → Server

Event	Payload	Description
`driver.location.update`	`{ driverId, lat, lon, heading, speed, status }`	Location heartbeat (every 3-5s). `status` : `available` / `assigned` / `in_trip`
`driver.ride.accept`	`{ driverId, tripRequestId }`	Driver accepts an offered ride
`driver.ride.decline`	`{ driverId, tripRequestId }`	Driver declines an offered ride
`driver.ride.arrived`	`{ tripId }`	Driver arrived at pickup location
`driver.ride.pickup`	`{ tripId }`	Rider picked up, trip starts
`driver.ride.dropoff`	`{ tripId }`	Rider dropped off, trip ends

User → Server

Event	Payload	Description
`user.ride.request`	`{ userId, sourceLocation, targetLocation }`	Request a new ride

Server → User

Event	Payload	Description
`user.ride.ack`	`{ tripRequestId }`	Immediate ack after ride request persisted
`user.ride.assigned`	`{ tripRequestId, tripId, driver, pickupETA, dropoffETA }`	Driver matched and assigned
`user.ride.driverLocation`	`{ tripId, location }`	Real-time driver location during trip
`user.ride.update`	`{ tripId, status }`	Status change: `driverArrived` / `start` / `completed`
`user.ride.noDriverFound`	`{ tripRequestId }`	No driver available (SLA exceeded)

Server → Driver

Event	Payload	Description
`driver.ride.request`	`{ driverId, tripRequestId, sourceLocation, targetLocation, fare, expiresAt }`	Ride offer (wave-based, expires in 2-5s)
`driver.ride.assigned`	`{ driverId, tripRequestId, tripId }`	Confirmed assignment after accept
`driver.ride.offerClosed`	`{ driverId, tripRequestId }`	Offer closed (another driver accepted)

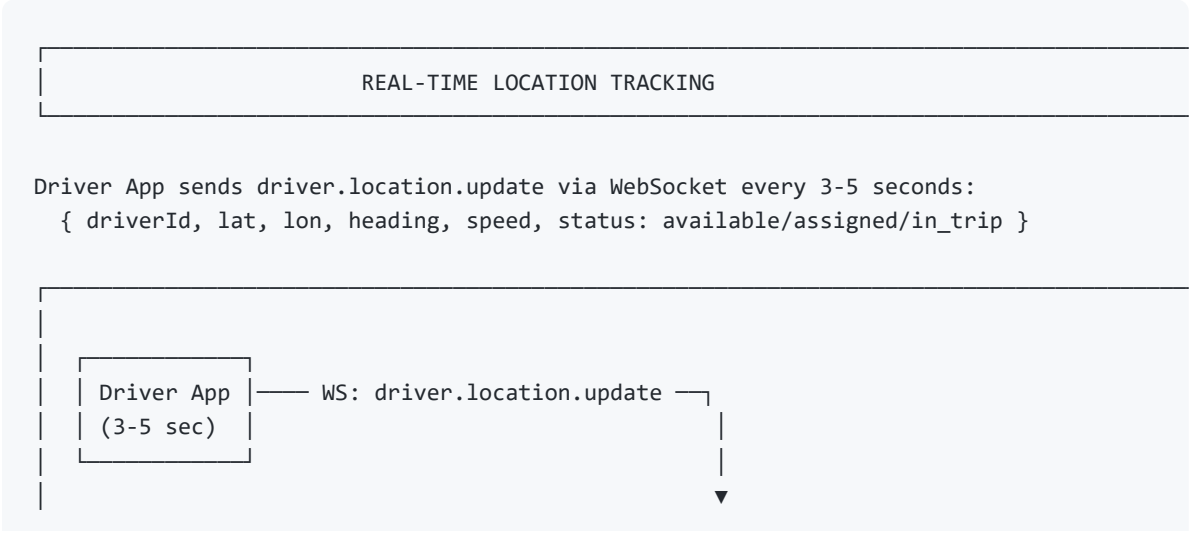
REST APIs

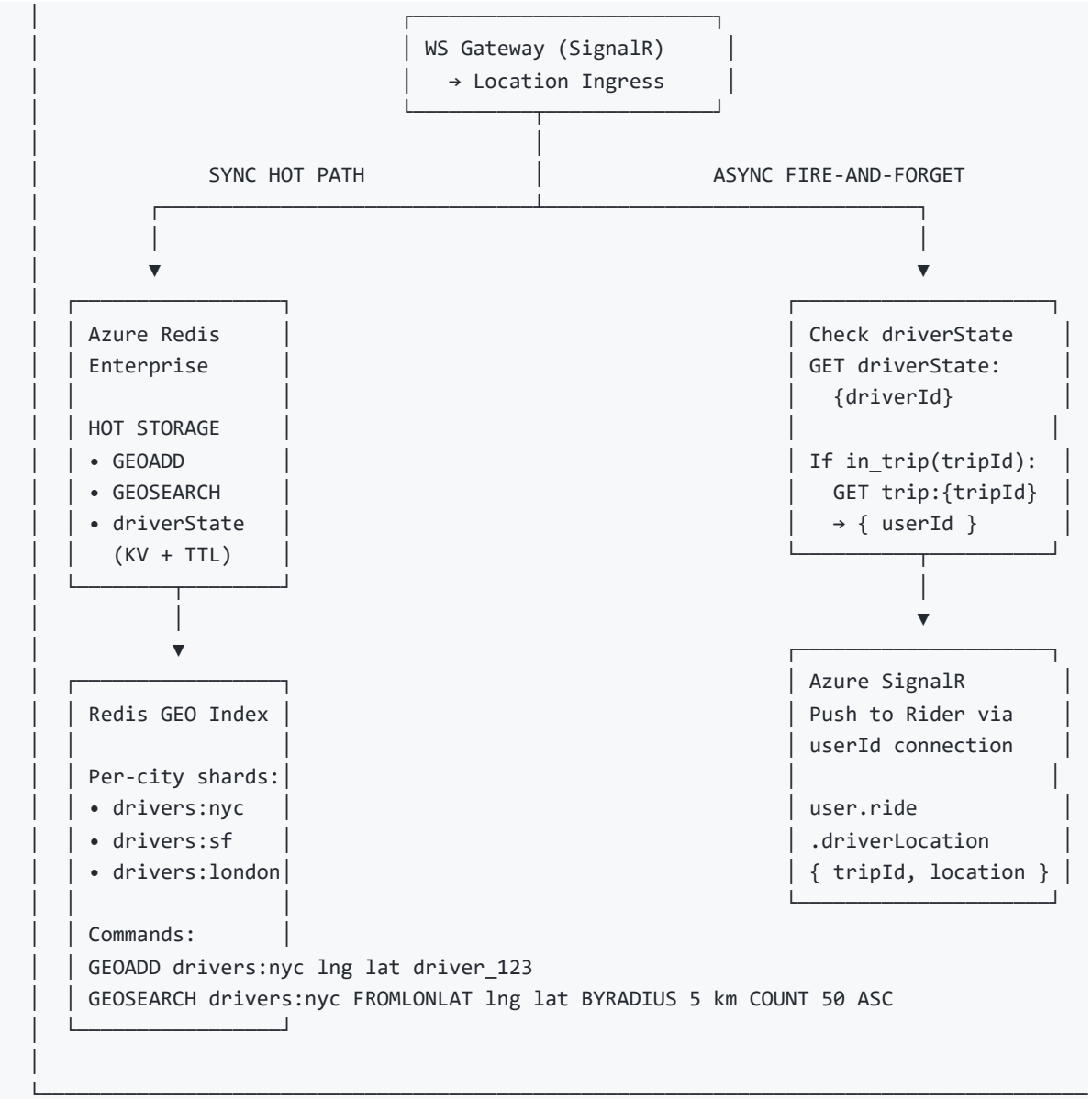
Payment

Method	Endpoint	Request	Response
`POST`	`/api/v1/user/payment/authorize`	`{ tripId, paymentMethodToken }`	`{ authorizationId, status }`
`POST`	`/api/v1/user/payment/capture`	`{ tripId, authorizationId }`	`{ receiptId, status }`

2) Real-Time Location Tracking (HOT PATH)

Architecture





Scale:

- 2M drivers × 1 update/3 sec = 667K writes/sec
- Redis Enterprise: 1M+ ops/sec per cluster
- Shard by city/geohash prefix

> 💡 **Why WebSocket, not HTTP polling?** At 667K updates/sec, HTTP polling would mean 667K TCP connections/sec (handshake overhead). WebSocket keeps one persistent connection per driver — orders of magnitude more efficient.

> 💡 **Why fire-and-forget for rider push?** The hot path (Redis write) must return in <5ms to keep up with 667K/sec. If SignalR push is slow or the rider's connection is flaky, it must not block the location write. The side path is async — dropping one push out of hundreds is invisible to the rider.

Location Update Flow

Step	From	To	Action	Sync/Async
1	Driver App	WS Gateway (SignalR)	`driver.location.update` via WebSocket	SYNC
2	WS Gateway	Location Ingress	Validate, transform	SYNC
3	Location Ingress	Redis GEO + KV	GEOADD to city shard + SET driverState (TTL)	SYNC
4	Location Ingress	Redis KV	**Fire-and-forget:** GET `driverState:{driverId}` — check if `in_trip(tripId)`	ASYNC
5	Location Ingress	Redis KV	If in_trip: GET `trip:{tripId}` → resolve `userId`	ASYNC
6	Location Ingress	SignalR → User App	Push `user.ride.driverLocation { tripId, location }` to rider's connection by `userId`	ASYNC

Redis GEO Commands

```
# Add/Update driver location
GEOADD "drivers:nyc" -73.985428 40.748817 "driver_123"

# Find nearest drivers within 5km radius
GEOSEARCH "drivers:nyc" FROMLONLAT -73.985428 40.748817 BYRADIUS 5 km COUNT 50 ASC W

# Result:
# 1) "driver_456" "0.3"   (300 meters)
# 2) "driver_789" "0.8"   (800 meters)
# 3) "driver_321" "1.2"   (1.2 km)

# Remove driver (went offline)
ZREM "drivers:nyc" "driver_123"

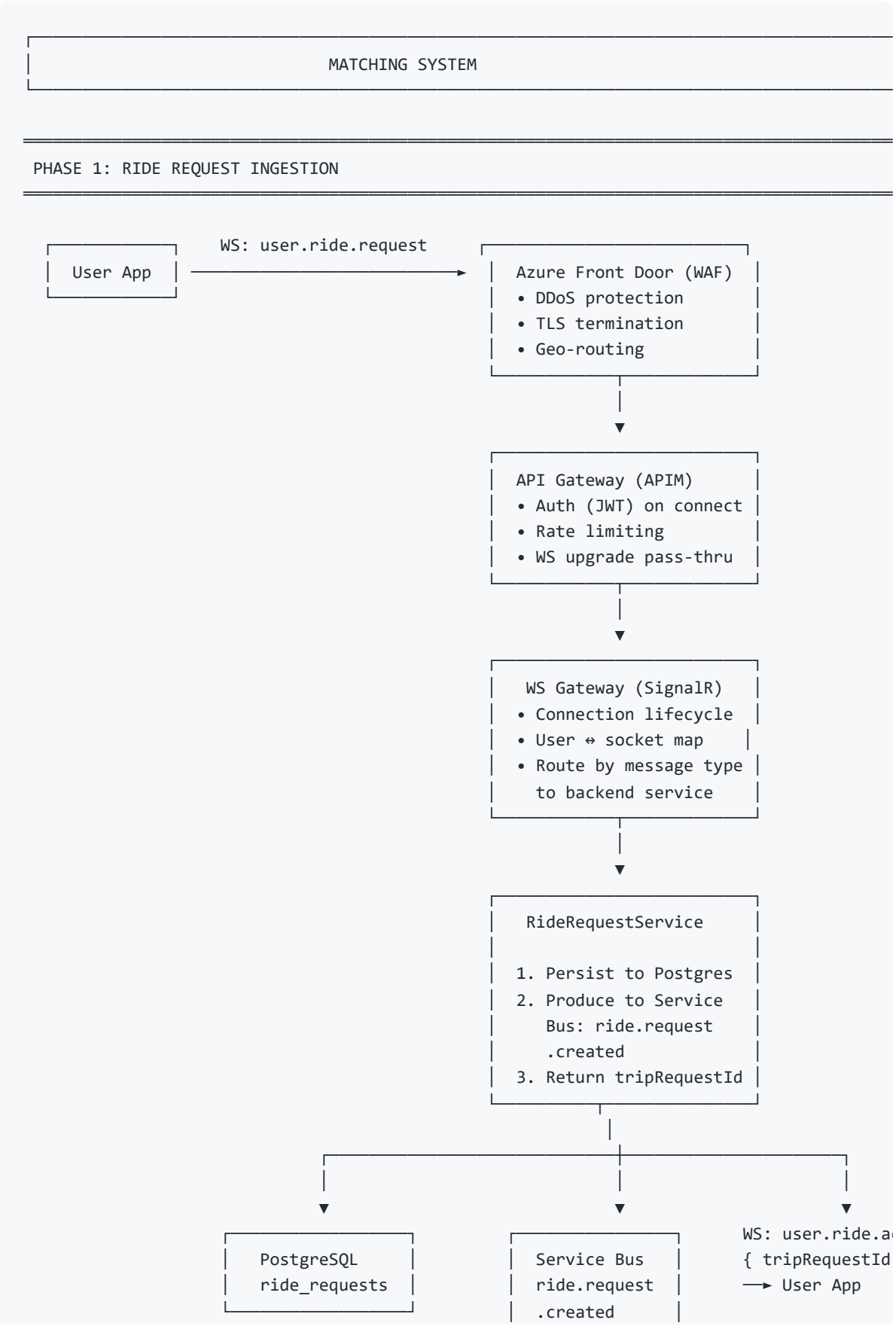
# Trip-to-rider mapping (set when trip is created, TTL = max trip duration)
SET "trip:trip_abc" '{"userId":"user_456","driverId":"driver_123"}' EX 7200

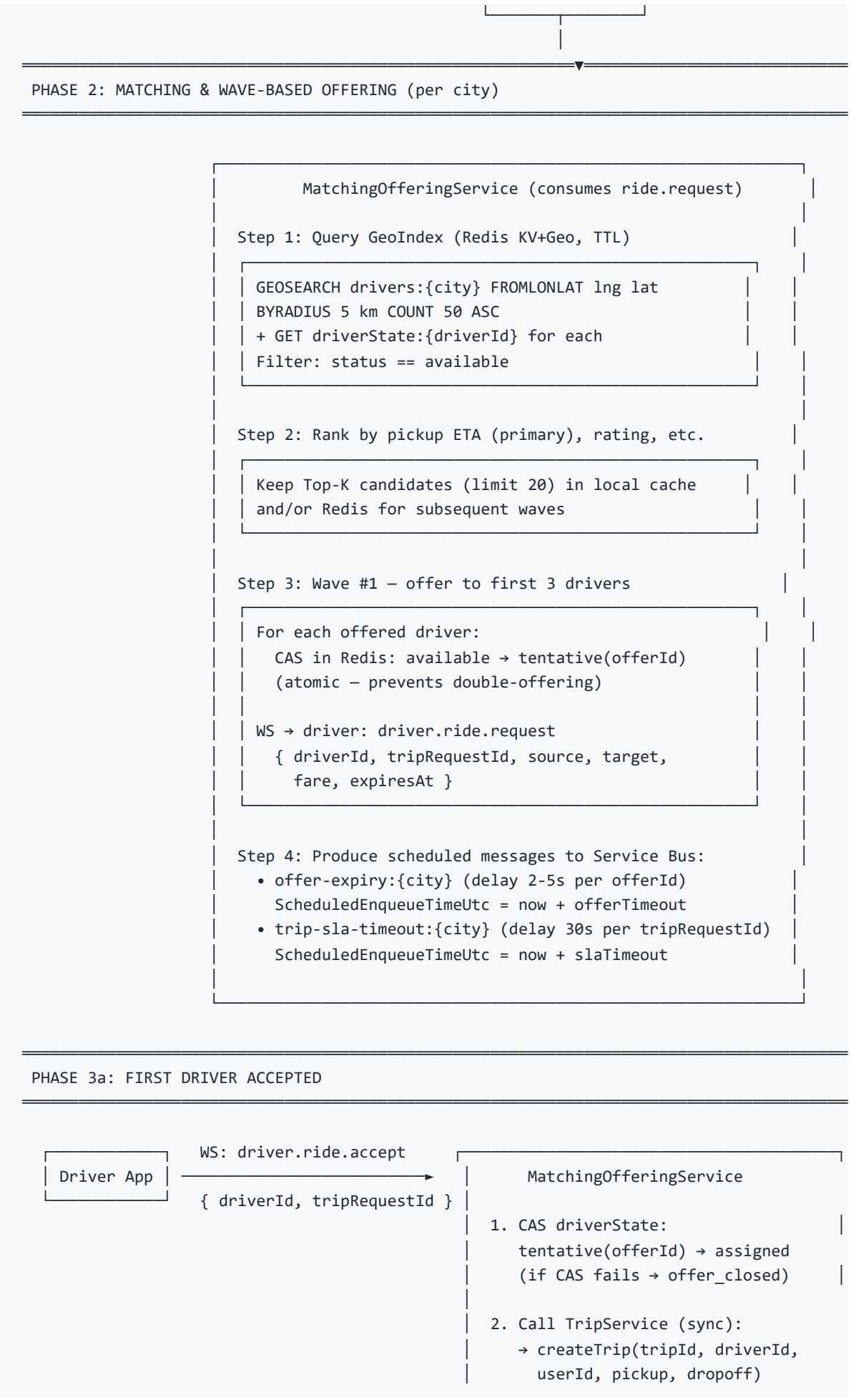
# Location push routing: check if driver is in active trip
GET "driverState:driver_123"          # → "in_trip:trip_abc"
GET "trip:trip_abc"                    # → {"userId":"user_456"} → push to user_456
```

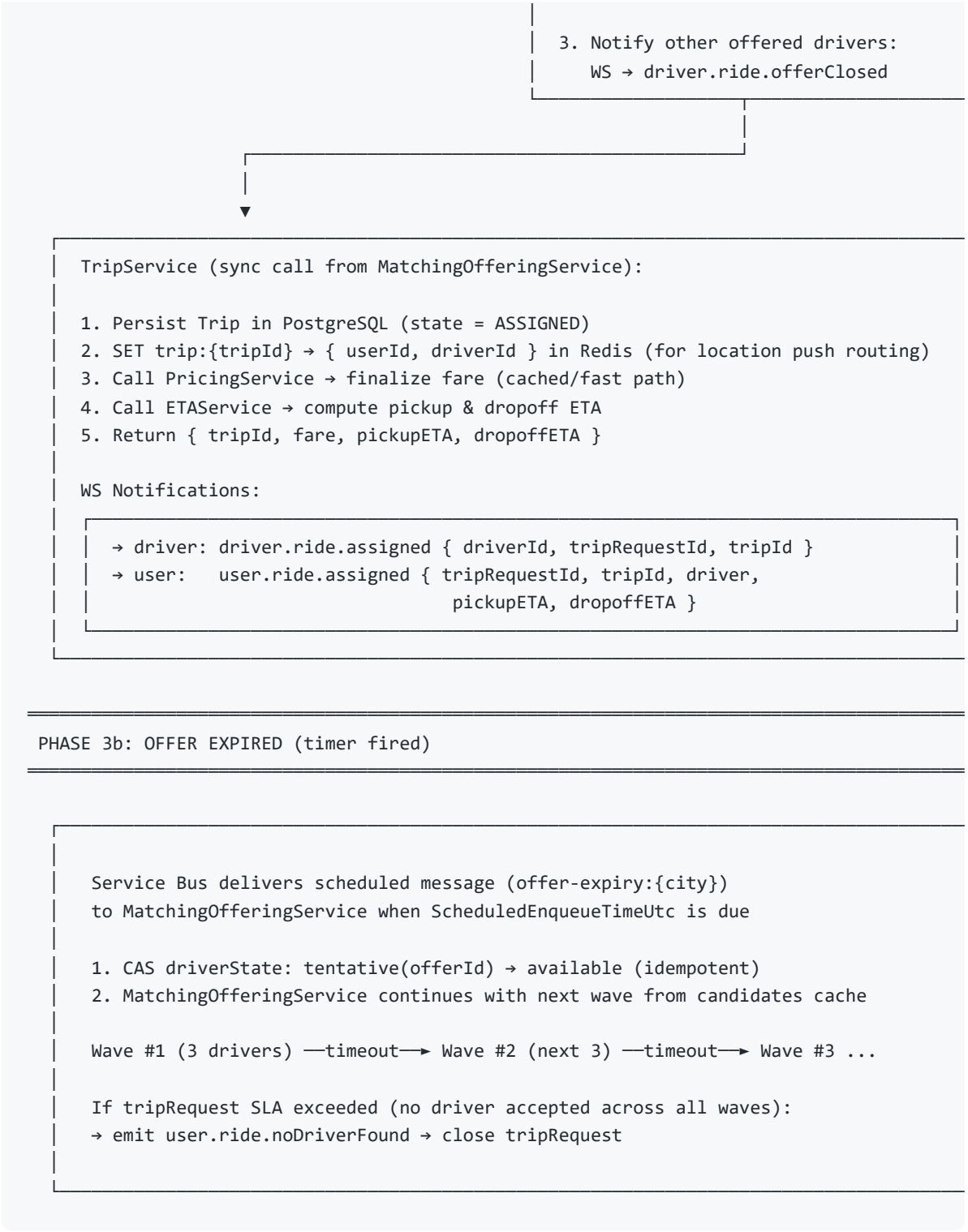
---

3) Matching System (City-Local, Wave-Based Offering)

Architecture




PHASE 2: MATCHING & WAVE-BASED OFFERING (per city)
MatchingOfferingService (consumes ride.request)
Step 1: Query GeoIndex (Redis KV+Geo, TTL)
GEOSEARCH drivers:{city} FROMLONLAT lng lat
BYRADIUS 5 km COUNT 50 ASC
+ GET driverState:{driverId} for each
Filter: status == available
Step 2: Rank by pickup ETA (primary), rating, etc.
Keep Top-K candidates (limit 20) in local cache
and/or Redis for subsequent waves
Step 3: Wave #1 — offer to first 3 drivers
For each offered driver:
CAS in Redis: available → tentative(offerId)
(atomic — prevents double-offering)
WS → driver: driver.ride.request
{ driverId, tripRequestId, source, target,
fare, expiresAt }
Step 4: Produce scheduled messages to Service Bus:
• offer-expiry:{city} (delay 2-5s per offerId)
ScheduledEnqueueTimeUtc = now + offerTimeout
• trip-sla-timeout:{city} (delay 30s per tripRequestId)
ScheduledEnqueueTimeUtc = now + slaTimeout
PHASE 3a: FIRST DRIVER ACCEPTED
Driver App
WS: driver.ride.accept
{ driverId, tripRequestId }
MatchingOfferingService
1. CAS driverState:
tentative(offerId) → assigned
(if CAS fails → offer_closed)
2. Call TripService (sync):
→ createTrip(tripId, driverId,
userId, pickup, dropoff)



Step-by-Step Flow: Ride Request Ingestion (Phase 1)

Step	From	To	Action	Data
1	User App	Azure Front Door	WS connection via Anycast	`user.ride.r`
2	Front Door	API Gateway (APIM)	WAF + DDoS filtering, TLS	Same payloa

			termination, geo-route	
3	API Gateway	WS Gateway (SignalR)	Auth (JWT) on connect, rate limit, WS upgrade pass-thru	Same payloa context
4	WS Gateway	RideRequestService	Route by message type (`user.ride.request`)	`{ userId, sourceLocat targetLocat`
5	RideRequestService	PostgreSQL	Persist ride request (within tx)	INSERT `ride_reques`
6	RideRequestService	Service Bus	Produce event (after tx commit)	`ride.reques { tripReque userId, pic dropoff, ci`
7	WS Gateway	User App	Ack immediately after tx commit	`user.ride.a tripRequest`

Step-by-Step Flow: Matching & Wave Offering (Phase 2)

Step	From	To	Action
1	Service Bus	MatchingOfferingService	Consume `ride.request.created` (per city)
2	MatchingOfferingService	Redis GEO + KV	GEOSEARCH nearby + GET driverState for each
3	MatchingOfferingService	—	Rank by pickup ETA (primary), rating, acceptance rate; keep Top-20 in cache
4	MatchingOfferingService	Redis	CAS per wave driver: `available → tentative(offerId)`

			(atomic, prevents double-offering)
5	MatchingOfferingService	WS Gateway → Driver Apps	Send offers to wave (3 drivers)
6	MatchingOfferingService	Service Bus (scheduled)	Produce scheduled messages: offer-expiry (delay 2-5s) + trip-sla-timeout (delay 30s) via `ScheduledEnqueueTimeUtc`

Step-by-Step Flow: First Driver Accepted (Phase 3a)

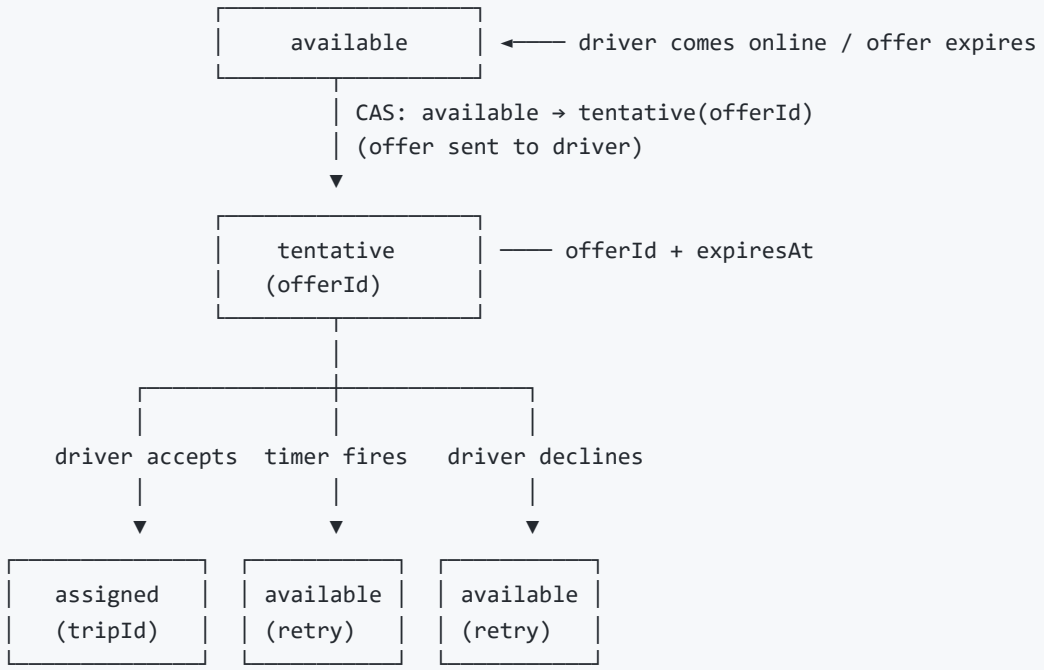
Step	From	To	Action	D
1	Driver App	WS Gateway	Accept ride offer	`{ dr }`
2	WS Gateway	MatchingOfferingService	Route by message type	S
3	MatchingOfferingService	Redis	CAS: `tentative(offerId) → assigned(tripId)` (if fail → offer_closed)	`{`
4	MatchingOfferingService	TripService	Call `createTrip` (sync)	`{ us }`
4a	TripService	PostgreSQL	Persist Trip (state = ASSIGNED)	IN
4b	TripService	Redis	SET `trip:{tripId}` → `{ userId, driverId }` (for location push routing)	T

4c	TripService	PricingService	Finalize fare (cached/fast path)	C e
4d	TripService	ETAService	Compute pickup + dropoff ETA	{ }
5	MatchingOfferingService	WS Gateway → other drivers	Notify: offer closed / assigned elsewhere	{ t
5b	MatchingOfferingService	Redis	CAS other offered drivers: `tentative(offerId) → available`	P
6	WS Gateway	Driver App	Assignment confirmation	d t
7	WS Gateway	User App	Full match notification	t d d

Step-by-Step Flow: Offer Expired (Phase 3b)

Step	From	To	Action
1	Service Bus (scheduled)	MatchingOfferingService	Deliver offer-expiry message when `ScheduledEnqueueTimeUtc` is due
2	MatchingOfferingService	Redis	CAS: `tentative(offerId) → available` (idempotent)
3	MatchingOfferingService	—	Continue with next wave from candidates cache (Top-20)
4	MatchingOfferingService	WS Gateway → User App	**If SLA exceeded:** no driver found, close tripRequest

Driver State Machine (Redis CAS)



Key Design Decisions

> 💡 **Why Redis GEO, not PostGIS?** PostGIS gives richer spatial queries but runs on disk. Redis GEO (`GEOADD` / `GEOSEARCH`) runs entirely in-memory at **1M+ ops/sec** — critical for the 667K writes/sec location hot path. The trade-off: Redis is ephemeral (no durability), but location data is transient by nature — a 3-second-old location is already stale.

> 💡 **Why CAS (Compare-And-Swap), not distributed locks?** A distributed lock (e.g., Redlock) requires acquiring and releasing — if the process crashes while holding the lock, the driver is stuck. CAS is atomic and stateless: `available → tentative(offerId)` succeeds or fails in one call. No lock cleanup needed. If the process crashes, the offer-expiry timer (Service Bus scheduled message) resets the driver to `available` automatically.

> 💡 **Why wave-based offering (3 drivers), not broadcast to all?** Broadcasting to 20 drivers creates a thundering herd — 19 drivers get "offer closed" immediately after 1 accepts. Waves of 3 give each driver a fair window (2-5s), reduce wasted WS messages, and the Top-K cache means we don't re-query Redis between waves.

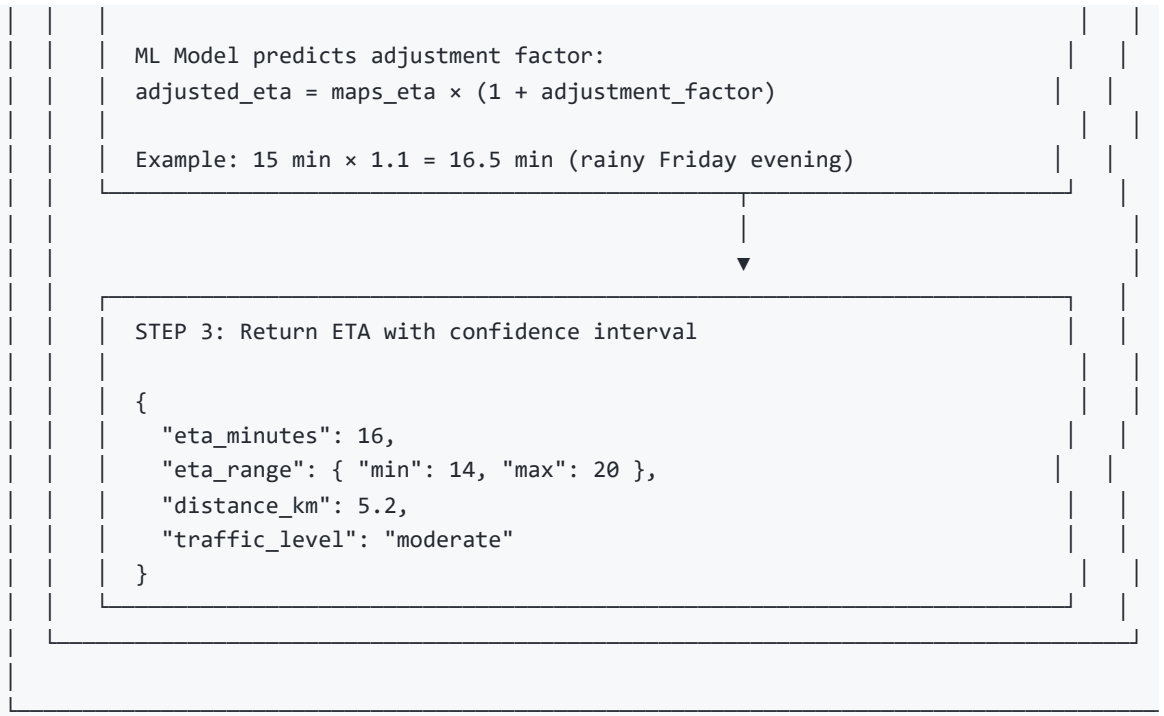
> 💡 **Why Service Bus (not Event Hubs) for ride events?** Ride requests are ~116/sec (not 667K/sec like locations). Service Bus handles this easily per city queue. The killer feature: **ScheduledEnqueueTimeUtc** — native delayed delivery for offer expiry and SLA timers. Event Hubs has no scheduling mechanism, so you'd need a separate timer infrastructure.

> 💡 **Why sync TripService call (not Service Bus event)?** The rider is waiting. An async `trip.created` event means eventual consistency — the rider might wait extra seconds for TripService to consume, persist, and compute ETA. At ~116 trips/sec, a sync call adds <50ms and gives the rider instant confirmation with fare and ETA.

4) ETA Service (Azure Maps + ML)

Architecture





Azure Maps APIs Used:

- Route API: Directions with real-time traffic
- Traffic Flow API: Current traffic conditions on road segments
- Weather API: Rain, snow, fog affecting travel
- Timezone API: Handle cross-timezone rides
- Geocoding API: Address to lat/lng conversion

Step-by-Step Flow

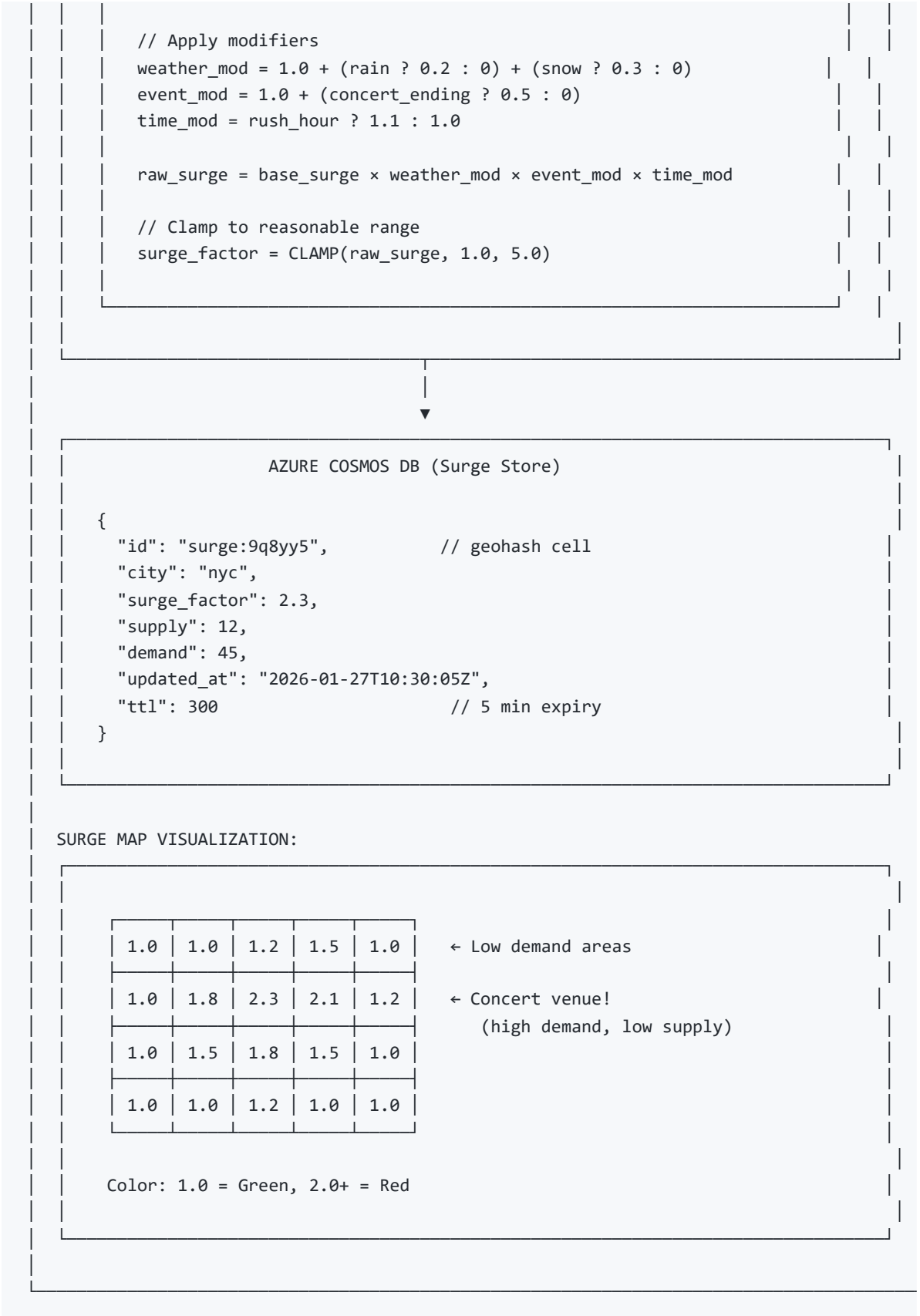
Step	From	To	Action	Data
1	Caller (MatchingOfferingService / PricingService)	ETA Service	Request ETA	`{ origin: {lat, lng}, destination: {lat, lng} }`
2	ETA Service	Azure Maps Route API	Get base travel time + traffic delay	`travelTimeInSeconds: 720, trafficDelayInSeconds: 180`
3	ETA Service	Azure ML Endpoint	Get ML adjustment factor	Features: `{ time_of_day, weather, historical_accuracy, events, surge }` → `adjustment_factor: 0.1`

4	ETA Service	—	Compute adjusted ETA	`adjusted_eta = maps_eta × (1 + adjustment_factor)` = 16.5 min
5	ETA Service	Caller	Return ETA with confidence	`{ eta_minutes: 16, eta_range: {min:14, max:20}, distance_km: 5.2, traffic_level: "moderate" }`

5) Surge Pricing System

Architecture





Step-by-Step Flow

Step	From	To	Action	Data

1	Azure Functions (timer: 5-10s)	Redis GEO + PostgreSQL	Query supply (GEOSEARCH per geohash cell) + demand (ride_requests count)	`{ cell, supply, demand }`
2	Stream Analytics	—	Tumbling window (5-10s) per geohash cell (precision 6 ≈ 600m)	Aggregate `supply` and `demand` per cell
3	Stream Analytics	—	Calculate surge: `(demand / supply) × weather_mod × event_mod × time_mod`, clamp to [1.0, 5.0]	`{ surge_factor, supply, demand }`
4	Stream Analytics	Cosmos DB	Write surge factor for cell (TTL: 5 min)	`{ cell: "surge:9q8yy5", surge_factor: 2.3, ttl: 300 }`
5	RideRequestService / PricingService	Cosmos DB	Read surge factor for fare estimate/finalization	`{ cell } → { surge_factor }`

---

6) Payment System (Strongly Consistent)

> 💡 **Why PostgreSQL (CP), not Cosmos DB (AP)?** Money requires strong consistency. If a payment capture is acknowledged, it must be durable — no "eventual consistency" for financial transactions. PostgreSQL gives ACID guarantees. Cosmos DB's eventual consistency could result in double-charges or lost payouts.

> 💡 **Why authorize/capture (two-phase), not charge-on-complete?** Authorize on assignment validates the rider can pay *before* the driver drives. If the card is declined, we cancel the match immediately — not after a 30-minute trip.

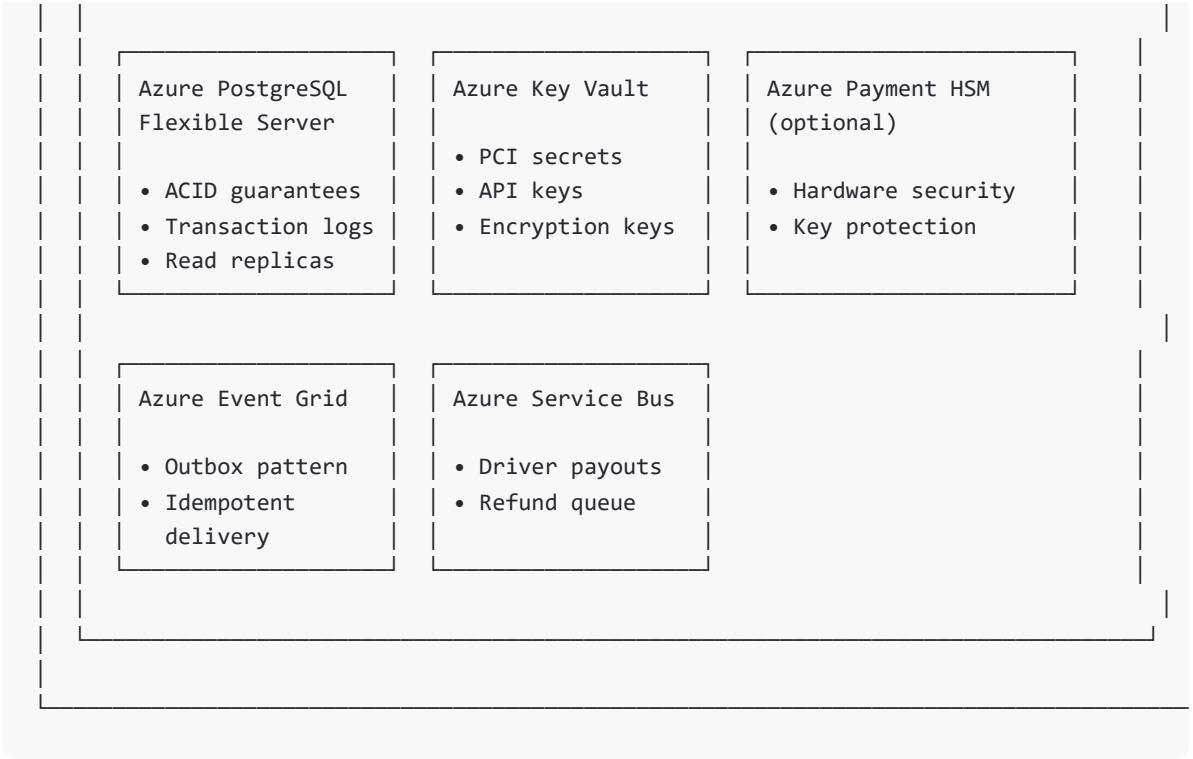
Architecture

```
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                                PAYMENT SYSTEM (CP - Strongly Consistent)
```

```
└──────────────────────────────────────────────────────────────────────────────────────────────┘

This MUST be CP (consistent + partition-tolerant). Use Azure PostgreSQL.

┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                                                                                              │
│  PAYMENT FLOW:                                                                               │
│                                                                                              │
│  ┌──────────────────────────────────────────────────────────────────────────────────────┐    │
│  │                                                                                      │    │
│  │  1. TRIP ASSIGNED → Authorize payment (hold estimated amount)                        │    │
│  │  ┌──────────────────────────────────────────────────────────────────────────────┐    │    │
│  │  │  POST /api/v1/user/payment/authorize                                         │    │    │
│  │  │  Request: { tripId, paymentMethodToken }                                     │    │    │
│  │  │  Response: { authorizationId, status }                                       │    │    │
│  │  │                                                                              │    │    │
│  │  │  INSERT INTO transactions (trip_id, amount, status, type)                    │    │    │
│  │  │  VALUES ('trip_123', 25.00, 'authorized', 'preauth')                         │    │    │
│  │  └──────────────────────────────────────────────────────────────────────────────┘    │    │
│  │                                ↓                                                     │    │
│  │  2. TRIP COMPLETED → Capture actual amount                                           │    │
│  │  ┌──────────────────────────────────────────────────────────────────────────────┐    │    │
│  │  │  POST /api/v1/user/payment/capture                                           │    │    │
│  │  │  Request: { tripId, authorizationId }                                        │    │    │
│  │  │  Response: { receiptId, status }                                             │    │    │
│  │  │                                                                              │    │    │
│  │  │  Final fare calculated:                                                      │    │    │
│  │  │    base_fare + (distance × rate) + (time × rate) × surge_factor              │    │    │
│  │  │    = $5 + (8km × $1.50) + (20min × $0.25) × 1.5 = $27.75                     │    │    │
│  │  │                                                                              │    │    │
│  │  │  BEGIN TRANSACTION;                                                          │    │    │
│  │  │    UPDATE transactions SET amount = 27.75, status = 'captured'               │    │    │
│  │  │    WHERE trip_id = 'trip_123';                                               │    │    │
│  │  │                                                                              │    │    │
│  │  │    INSERT INTO driver_payouts (driver_id, ride_id, amount)                   │    │    │
│  │  │    VALUES (456, 123, 22.20);  // 80% to driver                               │    │    │
│  │  │                                                                              │    │    │
│  │  │    INSERT INTO outbox (event_type, payload)                                  │    │    │
│  │  │    VALUES ('payment_captured', '{ride_id: 123, amount: 27.75}');             │    │    │
│  │  │  COMMIT;                                                                     │    │    │
│  │  │                                                                              │    │    │
│  │  │  Call Payment Gateway: capture(auth_code, $27.75)                            │    │    │
│  │  └──────────────────────────────────────────────────────────────────────────────┘    │    │
│  │                                ↓                                                     │    │
│  │  3. ASYNC: Process outbox events                                                     │    │
│  │  ┌──────────────────────────────────────────────────────────────────────────────┐    │    │
│  │  │  Azure Event Grid polls outbox table                                         │    │    │
│  │  │  → Emit 'payment_captured' event                                             │    │    │
│  │  │  → Service Bus: Queue payout to driver (weekly batch)                        │    │    │
│  │  │  → Notification: Send receipt to rider                                       │    │    │
│  │  └──────────────────────────────────────────────────────────────────────────────┘    │    │
│  │                                                                                      │    │
│  └──────────────────────────────────────────────────────────────────────────────────────┘    │
│                                                                                              │
│  AZURE SERVICES:                                                                             │
│  ┌──────────────────────────────────────────────────────────────────────────────────────┐    │
```



Step-by-Step Flow

Authorization (on trip assignment):

Step	From	To	Action	Data	S
1	User App	Payment Service	`POST /user/payment/authorize`	`{ tripId, paymentMethodToken }`	S
2	Payment Service	PostgreSQL	INSERT transaction (preauth, pending)	`{ trip_id, amount, status: 'authorized' }`	S
3	Payment Service	Payment Gateway	Authorize hold on card	Estimated fare amount	S
4	Payment Service	User App	Return authorization	`{ authorizationId, status }`	S

Capture (on trip completion):

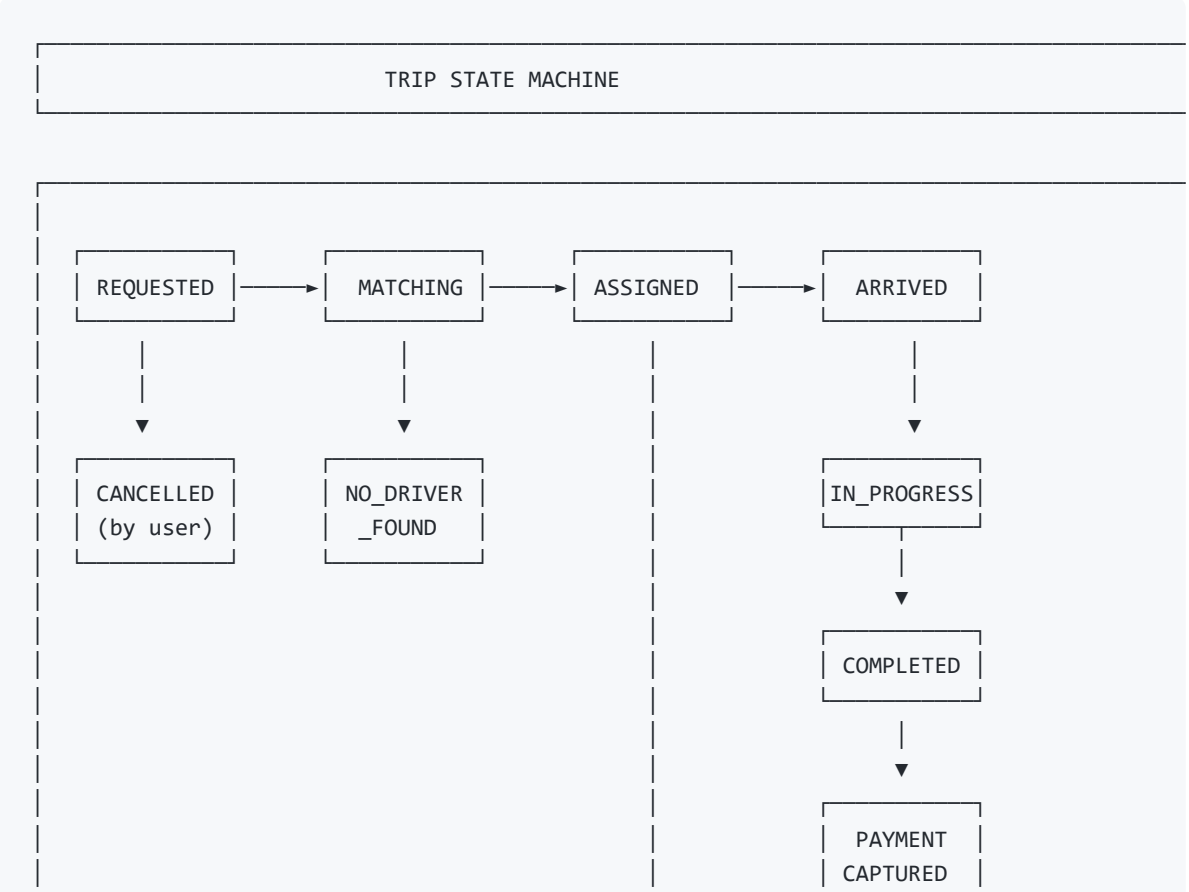
Step	From	To	Action	Data	Sync
5	User App	Payment Service	`POST /user/payment/capture`	`{ tripId, authorizationId }`	SYN

6	Payment Service	PricingService	Calculate final fare	`base + (dist × rate) + (time × rate) × surge`	SYN
7	Payment Service	PostgreSQL	BEGIN TX: update transaction + insert driver_payout + insert outbox	`{ amount: final, status: 'captured' }`	SYN
8	Payment Service	Payment Gateway	Capture actual amount	Final fare amount	SYN
9	Payment Service	User App	Return receipt	`{ receiptId, status }`	SYN
10	Outbox poller	Event Grid → Service Bus	Process outbox events	Driver payout queue + receipt notification	**ASY**

---

7) Ride Lifecycle Service (TripService)

State Machine



```
                                                  │            └──────────┘
│
│  Triggers:
│  ──────────────────────────────────────────────────────────────────────────────────────────
│  REQUESTED → MATCHING    : RideRequestService persists + produces ride.request.created
│  MATCHING  → ASSIGNED    : MatchingOfferingService: driver accepted, calls TripService
│  MATCHING  → NO_DRIVER   : SLA timer exceeded, no driver accepted across all waves
│  ASSIGNED  → ARRIVED     : driver.ride.arrived via WS
│  ARRIVED   → IN_PROGRESS : driver.ride.pickup via WS
│  IN_PROGRESS → COMPLETED : driver.ride.dropoff via WS
│  COMPLETED → PAYMENT_CAPTURED : POST /user/payment/capture succeeds
│
│  WS notifications on each transition:
│    → user:   user.ride.update { tripId, status }
│    → driver: driver.ride.assigned (on ASSIGNED only)
│
└──────────────────────────────────────────────────────────────────────────────────────────────

TRIP RECORD (PostgreSQL — persisted by TripService on sync call from MatchingOfferingSer
┌──────────────────────────────────────────────────────────────────────────────────────────────
│  {
│    "trip_id": "trip_abc123",
│    "trip_request_id": "req_xyz789",
│    "user_id": "user_456",
│    "driver_id": "driver_789",
│    "status": "ASSIGNED",
│    "pickup": { "lat": 40.7128, "lng": -74.0060 },
│    "dropoff": { "lat": 40.7580, "lng": -73.9855 },
│    "surge_factor": 1.5,
│    "estimated_fare": 25.00,
│    "final_fare": null,
│    "pickup_eta": "2026-01-27T10:08:00Z",
│    "dropoff_eta": "2026-01-27T10:30:00Z",
│    "timestamps": {
│      "requested_at": "2026-01-27T10:00:00Z",
│      "assigned_at": "2026-01-27T10:00:05Z",
│      "arrived_at": null,
│      "started_at": null,
│      "completed_at": null
│    }
│  }
└──────────────────────────────────────────────────────────────────────────────────────────────
```

Step-by-Step Flow

Step	Transition	Trigger (inbound)	Service	Ac
1	REQUESTED → MATCHING	`user.ride.request` via WS	RideRequestService	Pe pr `ri` to

2	MATCHING → ASSIGNED	`driver.ride.accept` via WS	MatchingOfferingService → TripService	CA Tri pe far
3	MATCHING → NO_DRIVER_FOUND	SLA timer exceeded	MatchingOfferingService	All clo
4	ASSIGNED → ARRIVED	`driver.ride.arrived { tripId }` via WS	TripService	Up
5	ARRIVED → IN_PROGRESS	`driver.ride.pickup { tripId }` via WS	TripService	Up sta
6	IN_PROGRESS → COMPLETED	`driver.ride.dropoff { tripId }` via WS	TripService	Up ca
7	COMPLETED → PAYMENT_CAPTURED	`POST /user/payment/capture`	PaymentService	Ca en

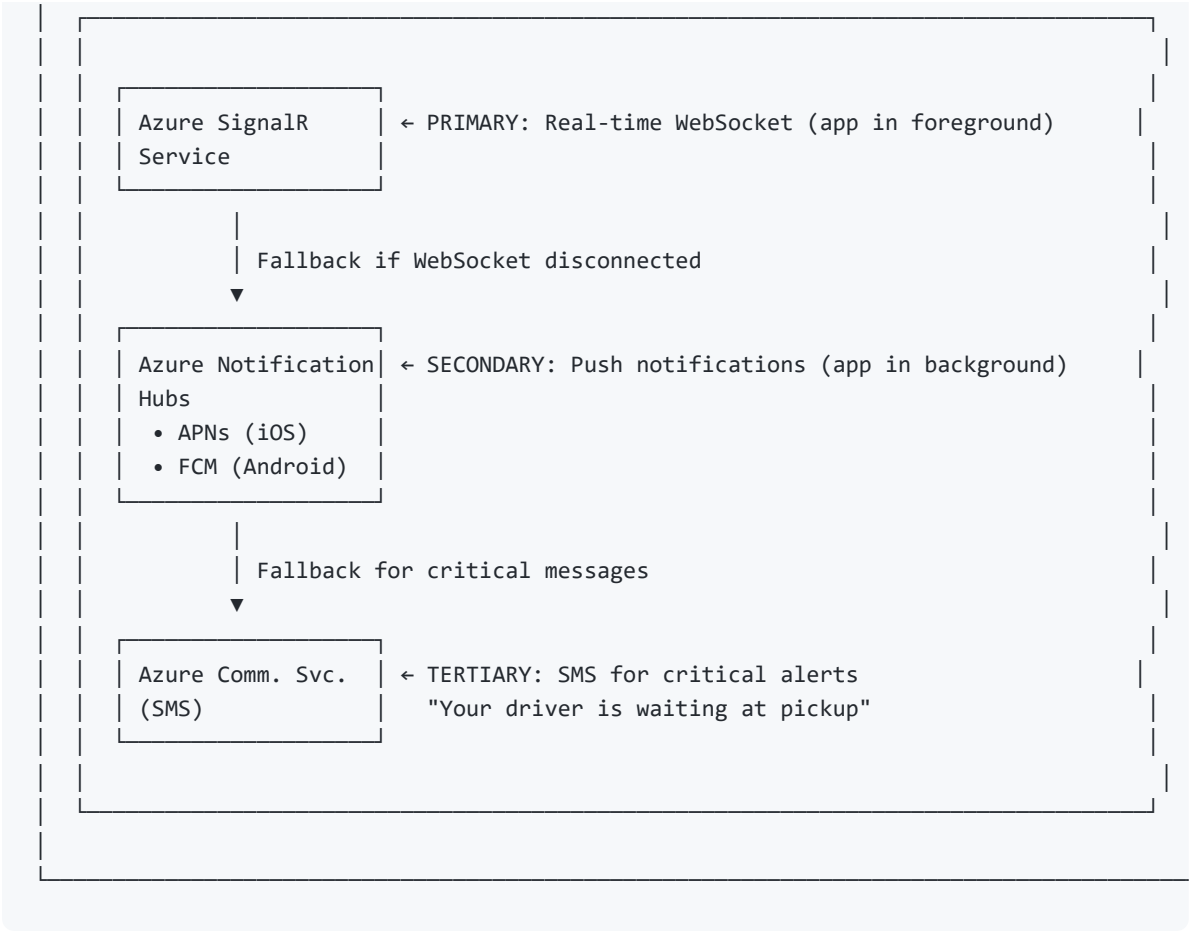
---

8) Notifications

Architecture

```
┌─────────────────────────────────────────────────────────────────────────────┐
│                          NOTIFICATION SYSTEM                                │
└─────────────────────────────────────────────────────────────────────────────┘


┌─────────────────────────────────────────────────────────────────────────────┐
│                                                                             │
│  NOTIFICATION TYPES:                                                        │
│  ┌──────────────────────────────────────────────────────────────────────┐   │
│  │  • Ride matched: "Your driver is on the way!"                        │   │
│  │  • Driver arrived: "Your driver has arrived at pickup"               │   │
│  │  • Ride completed: "Your trip is complete. Rate your driver"         │   │
│  │  • Driver: "New ride request!" (with accept/decline buttons)         │   │
│  │  • Promo: "50% off your next ride!" (marketing)                      │   │
│  └──────────────────────────────────────────────────────────────────────┘   │
│                                                                             │
│  DELIVERY CHANNELS:
```



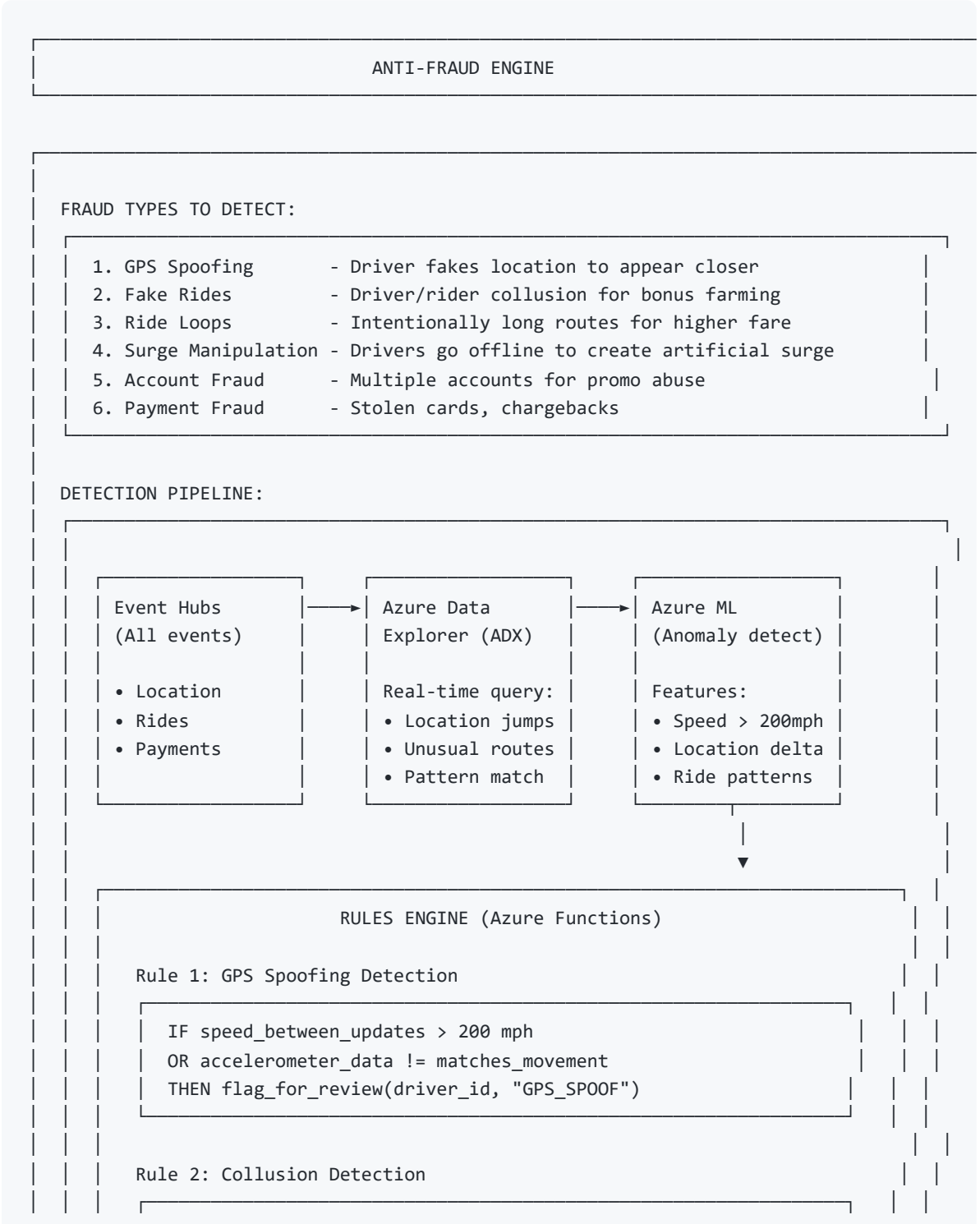
Step-by-Step Flow

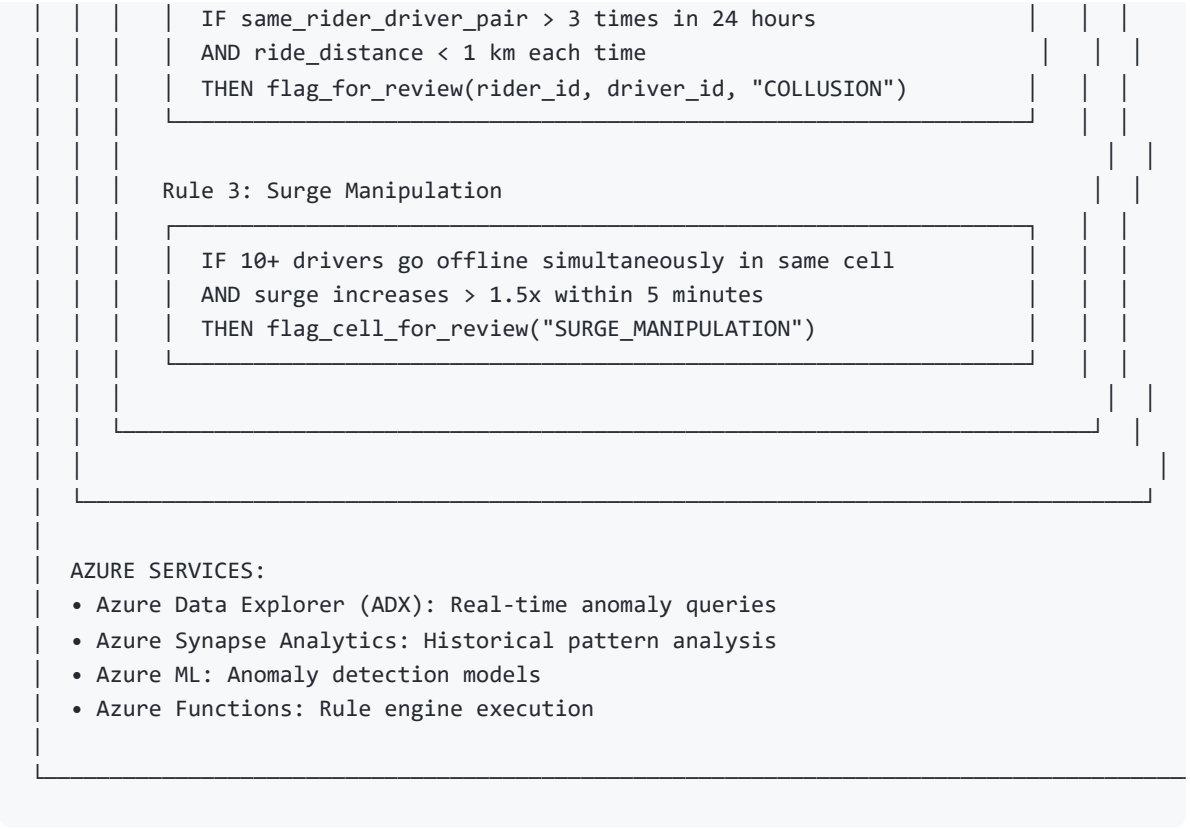
Step	From	To	Action	Data	Sync
1	Any Service	Notification Service	Emit notification event	`{ userId, type, channel[], template, data }`	ASYI
2	Notification Service	—	Check user WS connection state (SignalR connection registry)	`{ userId, isConnected }`	SYN
3a	Notification Service	Azure SignalR → User/Driver App	**If connected:** push via WebSocket	WS event (e.g., `user.ride.assigned`)	ASYI
3b	Notification Service	Azure Notification Hubs → APNs/FCM	**If disconnected:** push notification	`{ title, body, data }` → mobile push	ASYI

3c	Notification Service	Azure Communication Services → SMS	**If critical + no response:** send SMS	`"Your driver is waiting at pickup"`	ASYI

9) Anti-Fraud Engine

Architecture





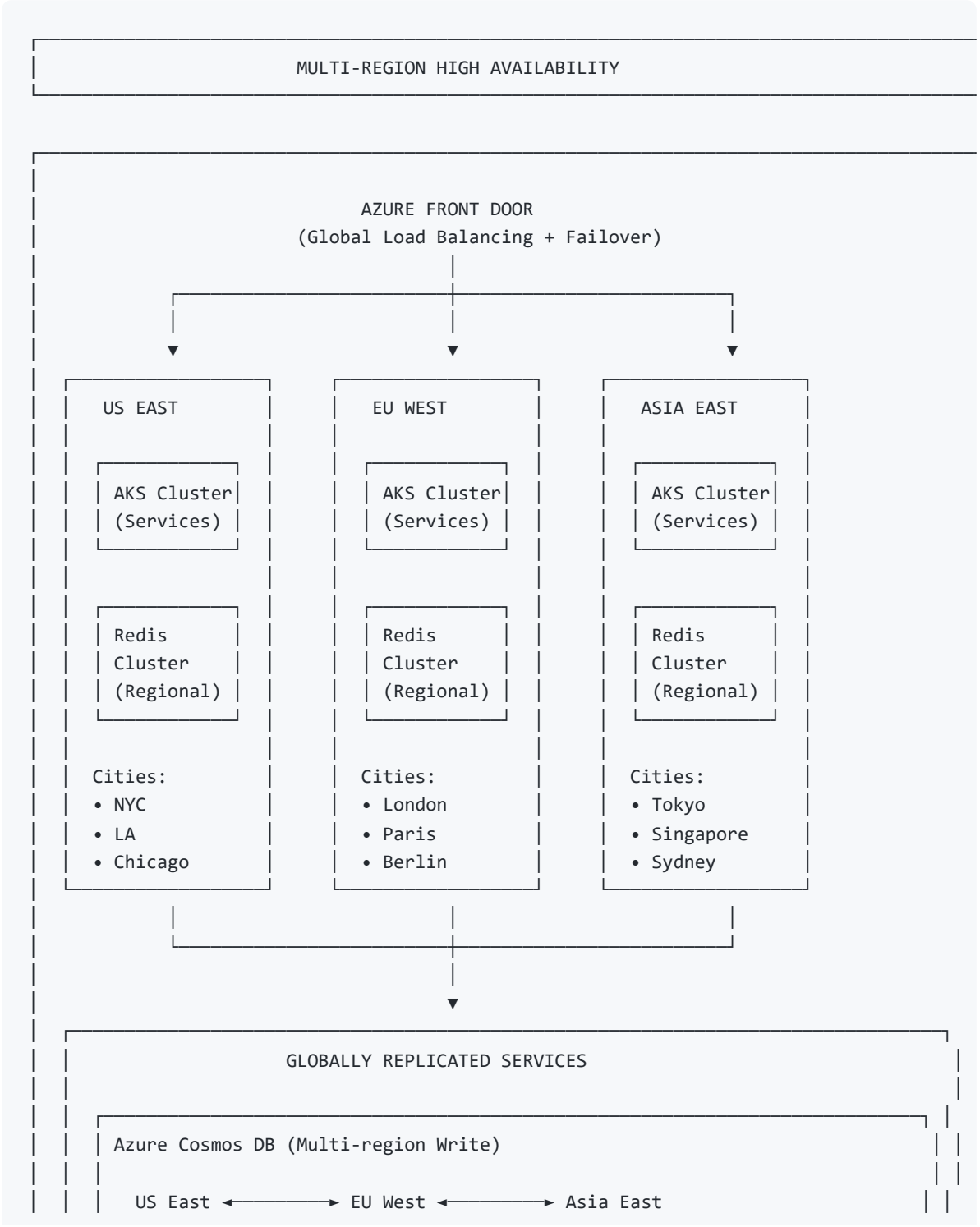
Step-by-Step Flow

Step	From	To	Action	Data	Sync/Async
1	All Services	Event Hubs	Emit events (location, rides, payments)	Raw event stream	ASYNC
2	Event Hubs	Azure Data Explorer (ADX)	Ingest events for real-time querying	Location jumps, unusual routes, patterns	ASYNC
3	ADX	Azure ML Endpoint	Extract features + anomaly scoring	`{ speed, location_delta, ride_patterns }`	SYNC
4	Azure ML	Rules Engine (Functions)	Evaluate fraud rules (GPS spoof, collusion, surge manipulation)	`{ anomaly_score, rule_matches[] }`	SYNC

5	Rules Engine	Action	Flag for review / block user / adjust payout	`{ driver_id, fraud_type, action }`	ASYNC

10) Multi-Region High Availability

Architecture





CITY ISOLATION STRATEGY:

11) Trade-offs Discussion

Decision	Option A	Option B	Choice	Reasoning
Location store	PostgreSQL PostGIS	**Redis GEO**	**Redis**	In-memory, 1M+ ops/sec, ephemeral data fits (see §2 💡)
Matching isolation	Global service	**Per-city cluster**	**Per-city**	Blast radius isolation, lower latency
Spatial index	QuadTree	**Geohash**	**Geohash**	Redis native, simpler, good enough for 5km radius
Driver offering	Broadcast to all	**Wave-based (3/wave)**	**Waves**	Less wasted WS messages, fair driver windows (see §3 💡)
Trip creation	Async via Service Bus	**Sync call to TripService**	**Sync**	Rider wants instant confirmation, ~116/sec fits (see §3 💡)
Offer expiry timers	Redis ZSET + timer worker	**Service Bus scheduled msgs**	**Service Bus**	ScheduledEnqueueTimeUtc — no extra infra (see §3 💡)
Payment consistency	Eventual (Cosmos DB)	**Strong (PostgreSQL ACID)**	**Strong**	Money requires strong consistency (see §6 💡)
ETA accuracy	Maps only	**Maps + ML**	**Maps + ML**	Azure Maps + ML gives ±2min accuracy
Surge compute	On-demand	**Pre-computed (poll Redis)**	**Pre-computed**	5-10 sec refresh is fast enough
Event streaming	Kafka	**Event Hubs**	**Event Hubs**	Native Azure, managed, Kafka API compatible

---

12) 60-Second Explanation

> "A ride-sharing platform is fundamentally a **real-time location-based matching system**. I'll design for 10M rides/day across 500 cities.
>
> **Location tracking** is the hot path: 2M drivers updating every 3 seconds = 700K writes/sec. I use **Azure Redis Enterprise with GEO commands** — `GEOADD` to store locations, `GEOSEARCH`

> to find nearby drivers in O(log N). During active trips, a fire-and-forget side path looks up the rider via `trip:{tripId}` in Redis and pushes location via **SignalR**.
>
> **Matching is city-isolated and wave-based**. When a rider requests, **RideRequestService** persists to Postgres and produces `ride.request.created` to **Service Bus** (partitioned by city). **MatchingOfferingService** consumes the event, queries Redis GEO for the nearest 50 drivers, ranks by pickup ETA, and offers to 3 drivers at a time (waves). Each offered driver's state is CAS'd in Redis: `available → tentative(offerId)`. Offer expiry uses **Service Bus scheduled messages** (`ScheduledEnqueueTimeUtc`, 2-5s delay) — no separate timer infrastructure needed.
>
> **When a driver accepts**, MatchingOfferingService CAS's the state to `assigned(tripId)` and calls **TripService synchronously** — TripService persists the trip, calls **PricingService** and **ETAService** internally, and returns `{ tripId, fare, pickupETA, dropoffETA }`. Rider gets instant confirmation via WebSocket.
>
> **Surge pricing** polls Redis GEO every 5-10s per geohash cell, calculates supply/demand ratio with weather and event modifiers, and stores in **Cosmos DB** with 5-min TTL.
>
> **Payments are strongly consistent** in **PostgreSQL** — authorize on assignment, capture on completion, outbox pattern for payout events.
>
> Multi-region with **Front Door** routing, city-level blast radius isolation, and **99.9% availability**."

Technologies Deep Dive

Technology	Role	Reference
Redis GEO	Real-time driver location with GEOADD/GEOSEARCH	Tech Guide
Geohash	Spatial indexing for surge pricing cells	Tech Guide
Kafka/Event Hubs	Event streaming for location and ride events	Tech Guide
PostgreSQL	ACID transactions for rides and payments	Tech Guide
Cosmos DB	Globally replicated driver status	Tech Guide
SignalR	Real-time WebSocket push to riders/drivers	Tech Guide

---

Azure Service Mapping Table

Problem	Azure Solution
Real-time locations	**Azure Redis Enterprise GEO**
Massive ingestion	**Azure Event Hubs**
Matching engine	**Azure Kubernetes Service (AKS)**
ETA calculation	**Azure Maps + Azure ML**
Surge pricing	**Stream Analytics + Azure Functions**
Payments	**Azure PostgreSQL + Key Vault**
Real-time updates	**Azure SignalR Service**
Push notifications	**Azure Notification Hubs**
Fraud detection	**Azure Synapse + ADX + ML**
Long-term history	**Azure Data Lake + Synapse**
Multi-region routing	**Azure Front Door**
Driver dispatch	**Azure Service Bus**

Appendix: Geohash Precision Reference

```
┌──────────────────────────────────────────────────────────┐
│                  GEOHASH PRECISION LEVELS                │
├─────────────┬──────────────────┬─────────────────────────┤
│ Precision   │ Cell Size        │ Use Case                │
├─────────────┼──────────────────┼─────────────────────────┤
│     4       │ ~20 km           │ Regional aggregation    │
│     5       │ ~5 km            │ City-level surge zones  │
│     6       │ ~600 m           │ ✅ Surge pricing cells (default) │
│     7       │ ~150 m           │ Block-level precision   │
│     8       │ ~20 m            │ Street-level precision  │
└─────────────┴──────────────────┴─────────────────────────┘

Example: Times Square, NYC
  Precision 4: dr5r
  Precision 6: dr5ru6
  Precision 8: dr5ru6hm
```

Appendix: Wave-Based Matching Pseudocode

```
WAVE_SIZE = 3
TOP_K = 20
OFFER_TTL_SEC = 5

def on_ride_request_created(ride_request):
    """Consumed from Service Bus: ride.request.created (per city)."""

    # Step 1: Query nearby candidates from GeoIndex (Redis KV+Geo, TTL)
    nearby = redis.geosearch(
        key=f"drivers:{ride_request.city}",
        longitude=ride_request.pickup_lng,
        latitude=ride_request.pickup_lat,
        radius=5, unit="km", count=50, sort="ASC"
    )

    # Step 2: Filter by driverState == available
    candidates = []
    for driver_id, distance in nearby:
        state = redis.get(f"driverState:{driver_id}")
        if state == "available":
            candidates.append((driver_id, distance))

    # Step 3: Rank by pickup ETA (primary), rating, acceptance_rate
    scored = []
    for driver_id, distance in candidates:
        eta = eta_service.get_pickup_eta(driver_id, ride_request.pickup)
        rating = redis.hget(f"driver:{driver_id}", "rating")
        scored.append(ScoredDriver(driver_id, eta, rating, distance))

    scored.sort(key=lambda d: d.eta)  # Primary: pickup ETA
    top_k = scored[:TOP_K]

    # Cache Top-K for subsequent waves
    candidate_cache.put(ride_request.trip_request_id, top_k)

    # Step 4: Start wave #1
    send_wave(ride_request, top_k, offset=0)

    # Step 5: Produce scheduled messages to Service Bus
    service_bus.send_scheduled(
        queue=f"trip-sla-timeout:{ride_request.city}",
        body={"tripRequestId": ride_request.trip_request_id},
        scheduled_enqueue_time=now() + ride_request.sla_timeout
    )

def send_wave(ride_request, candidates, offset):
    """Send offers to next WAVE_SIZE drivers."""
    wave = candidates[offset : offset + WAVE_SIZE]

    for driver in wave:
        offer_id = generate_uuid()

        # CAS in Redis: available → tentative(offerId) — atomic
        locked = redis.cas(
```

```
            key=f"driverState:{driver.driver_id}",
            expected="available",
            new_value=f"tentative:{offer_id}"
        )

        if locked:
            # Send offer via WS Gateway
            ws_gateway.send(driver.driver_id, {
                "event": "driver.ride.request",
                "driverId": driver.driver_id,
                "tripRequestId": ride_request.trip_request_id,
                "sourceLocation": ride_request.pickup,
                "targetLocation": ride_request.dropoff,
                "fare": ride_request.fare_estimate,
                "expiresAt": now() + OFFER_TTL_SEC
            })

            # Produce scheduled offer-expiry message to Service Bus
            service_bus.send_scheduled(
                queue=f"offer-expiry:{ride_request.city}",
                body={"offerId": offer_id, "driverId": driver.driver_id},
                scheduled_enqueue_time=now() + OFFER_TTL_SEC
            )

def on_driver_accepted(driver_id, trip_request_id):
    """driver.ride.accept arrives via WS Gateway."""
    offer_id = extract_offer_id(driver_id)

    # CAS: tentative(offerId) → assigned(tripId); if fail → offer_closed
    trip_id = generate_trip_id()
    success = redis.cas(
        key=f"driverState:{driver_id}",
        expected=f"tentative:{offer_id}",
        new_value=f"assigned:{trip_id}"
    )

    if not success:
        ws_gateway.send(driver_id, {"event": "driver.ride.offerClosed"})
        return

    # Call TripService (sync) — orchestrates Pricing + ETA internally
    trip_result = trip_service.create_trip(
        trip_id=trip_id,
        driver_id=driver_id,
        user_id=ride_request.user_id,
        pickup=ride_request.pickup,
        dropoff=ride_request.dropoff
    )
    # trip_result = { tripId, fare, pickupETA, dropoffETA }

    # Notify other offered drivers: offer_closed
    for other_driver in get_offered_drivers(trip_request_id):
        if other_driver != driver_id:
            redis.cas(f"driverState:{other_driver}", f"tentative:*", "available")
            ws_gateway.send(other_driver, {"event": "driver.ride.offerClosed"})
```

```
def on_offer_expired(offer_id, driver_id):
    """Service Bus delivers scheduled offer-expiry message when ScheduledEnqueueTime
    # CAS: tentative(offerId) → available (idempotent)
    redis.cas(
        key=f"driverState:{driver_id}",
        expected=f"tentative:{offer_id}",
        new_value="available"
    )

    # Continue with next wave from candidates cache
    trip_request_id = get_trip_request(offer_id)
    candidates = candidate_cache.get(trip_request_id)
    next_offset = get_next_offset(trip_request_id)

    if next_offset < len(candidates):
        send_wave(get_ride_request(trip_request_id), candidates, next_offset)

    # If tripRequest SLA exceeded → no driver found
    if sla_exceeded(trip_request_id):
        ws_gateway.send(get_user_id(trip_request_id), {
            "event": "user.ride.noDriverFound",
            "tripRequestId": trip_request_id
        })
        close_trip_request(trip_request_id)
```